Deploying the Claude Managed Agents Learning Platform to AWS

Start here

This kit follows the 13:22 Udemy video. Build the included application in AWS and open your own protected deployment. Use AWS CloudShell in your browser; you do not need to install AWS CLI or Docker on your computer.

The kit includes application source, existing course images and audio, package-lock.json, the Dockerfile, deployment templates and helper scripts. CodeBuild installs dependencies, so access to the Node package registry and container base image is required. The application source matches the successful recording. The student helper adds an explicit account configuration step before the video commands. Never copy the instructor's account ID or application URL.

1. Prerequisites and costs

- Use your own AWS learning account or an approved training sandbox.
- Check IAM-REQUIREMENTS.md with your account administrator.
- Select Europe (Stockholm), region eu-north-1.
- Have an existing default VPC, usable default subnets in at least two zones, and active internet-gateway routes. Preflight checks these without changing them.
- Have Fargate On-Demand quota for a one-vCPU, 2-GiB task. Other workloads can consume your quota; the value shown in the video is only an example.
- Choose your own spending estimate and decide when to delete the deployment.

The video used €10. That is not a price guarantee or hard spending cap. Review current Fargate, CodeBuild, ECR, S3, load balancer, public IPv4, logging and data transfer charges. Resources can keep billing until deletion completes.

CloudShell documents 1 GB of persistent home storage per region. Check space:

```
df -h "$HOME"
```

Leave room for this ZIP and small helper/state files. Do not extract the whole application into CloudShell; CodeBuild expands it in AWS.

2. Open CloudShell and check identity

Sign in to the AWS console, select Stockholm and open CloudShell. Complete MFA yourself. Keep passwords and tokens outside source files and screenshots.

```
export AWS_REGION=eu-north-1
export AWS_DEFAULT_REGION=eu-north-1
export AWS_PAGER=""
aws sts get-caller-identity
```

Inspect Account and Arn. Authentication does not prove every creation permission.

3. Upload the ZIP and extract only the helpers

Choose CloudShell Actions → Upload file. Upload the unchanged file named Claude-Managed-Agents-AWS-Student-Kit.zip. Then run:

```
mkdir -p ~/claude-aws
unzip -o ~/Claude-Managed-Agents-AWS-Student-Kit.zip 'aws/*' -d ~/claude-aws
cd ~/claude-aws
```

Keep the full ZIP in your home directory; do not expand the course media there. If you rename the upload, adjust every --source-zip path accordingly.

4. Configure your own account

This is the additional student step before the video's preflight command. Enter your own 12-digit account ID from get-caller-identity:

```
read -r -p "Your 12-digit AWS account ID: " STUDENT_AWS_ACCOUNT_ID
python3 aws/cloudshell-deploy.py configure --account-id "$STUDENT_AWS_ACCOUNT_ID"
```

Expected output: your account ID, eu-north-1 and awsResourcesCreated: false. The helper saves aws/deployment-config.json locally and rejects a different signed-in account on later commands. Do not change this file to switch accounts after starting a deployment. No instructor credentials are included.

5. Run preflight

```
python3 aws/cloudshell-deploy.py preflight
```

Expected: templateValidation: passed. The helper validates both templates, the default network/routes, subnets, CloudFront prefix list and Fargate quota. creationPermissionsVerified remains false because read checks cannot prove permission to create everything. Resolve exact errors before proceeding.

6. Create the build stack

Enter a positive number in euros, with a decimal point if necessary:

```
read -r -p "Your deployment budget estimate in EUR: " DEMO_BUDGET_EUR
python3 aws/cloudshell-deploy.py build \
  --source-zip ~/Claude-Managed-Agents-AWS-Student-Kit.zip \
  --approved-budget-eur "$DEMO_BUDGET_EUR"
python3 aws/cloudshell-deploy.py status
```

The first invocation creates the dedicated build infrastructure. Poll status until claude-learning-aws-video-build shows CREATE_COMPLETE. FAILED or ROLLBACK requires diagnosis. This budget argument records your estimate; it creates no AWS Budget, billing alert, automatic shutdown or spending cap.

7. Start the image build once

When the build stack is ready, run the same build command once more:

```
python3 aws/cloudshell-deploy.py build \
  --source-zip ~/Claude-Managed-Agents-AWS-Student-Kit.zip \
  --approved-budget-eur "$DEMO_BUDGET_EUR"
```

You should receive a buildId. The ZIP is uploaded to private S3; CodeBuild checks the source manifest, builds the image and pushes it to private ECR. Poll only:

```
python3 aws/cloudshell-deploy.py status
```

Wait for imageBuild.buildStatus: SUCCEEDED. BUILD and POST_BUILD are intermediate phases. Do not repeat build to poll. The cloud build continues if CloudShell disconnects; use its saved ID after returning to the same helper folder.

8. Deploy the successful image

```
python3 aws/cloudshell-deploy.py deploy --approved-budget-eur "$DEMO_BUDGET_EUR"
python3 aws/cloudshell-deploy.py status
```

The helper resolves the immutable image tag to its ECR digest and creates the separate app stack. Poll until all these conditions appear:

- App stack: CREATE_COMPLETE.
- Service: ACTIVE; desiredCount 1, runningCount 1, pendingCount 0.
- Load balancer target: healthy.
- cloudProvisioningComplete: true.

Your own CloudFront Origin is returned. Creation can take several minutes. Use status to poll; do not repeat deploy.

9. Verify HTTPS and access protection

```
python3 aws/cloudshell-deploy.py verify
```

Expected results:

- anonymous-app-denied: 401.
- health-ready: 200, readiness ready.
- authenticated-home: 200.
- foreign-origin-denied: 403.
- tutor-status: 200, live tutor unavailable.

httpVerification.complete should be true. These checks make no paid model calls. The evidence is saved in aws-state/evidence.json. A later status refreshes that file; run verify again to export current HTTP checks. Browser workflows remain a separate check.

10. Open your own application privately

Run this command away from any screen recording:

```
python3 aws/cloudshell-deploy.py access
```

Open the returned origin and enter the generated username/password. They are unique to your deployment. Keep aws-state/recording-access.json private. Never upload it to Udemy or Git, and retain it until cleanup is finished.

CloudFront supplies browser HTTPS without your own domain. Its ALB origin connection uses HTTP. A private origin header and CloudFront prefix list gate the load balancer. This is not an end-to-end TLS or production student identity design. Do not open the task port to the public internet to bypass an error.

11. Repeat the video exercises

- Open Home and click Check backend; inspect the ready response.
- Open Lab 01 → Run the exercise; run the normal offline fixture and confirm a completed turn outcome.
- Choose Stream ends early; confirm an incomplete outcome.
- Open TypeScript source; confirm the deployed teaching source is readable.
- Open AI Tutor; request prepared guidance. Confirm the answer is marked Prepared and live mode is disabled.
- Save a notebook note, reload, reopen the notebook and confirm the note remains.

Notebook reload proves browser-local state for that profile/hostname. It does not prove durable server storage. Task files and the local database are ephemeral in this template. Separate milestone servers and host integrations need their own setup. Paid provider execution is disabled by default.

12. Resume after a disconnect

```
cd ~/claude-aws
python3 aws/cloudshell-deploy.py status
```

Keep aws/deployment-config.json, aws-state and the uploaded ZIP. Do not delete state or extract another helper folder to force a retry. A new terminal loses shell variables; restore the saved budget if needed:

```
DEMO_BUDGET_EUR=$(python3 -c 'import json; print(json.load(open("aws-state/budget.json"))
["estimatedBudgetEuros"])')
```

If the first build failed before saving a budget, re-enter your estimate. Read TROUBLESHOOTING.md before retrying failed AWS operations.

13. Clean up when finished

Save any notes you need first. Cleanup permanently removes the dedicated task files, private images, source objects and logs. Existing VPC/subnets/routes are preserved. Enter the same account ID you configured:

```
read -r -p "Account ID whose lesson resources you want to delete: " STUDENT_AWS_ACCOUNT_ID
python3 aws/cloudshell-deploy.py cleanup \
  --confirm-delete-account "$STUDENT_AWS_ACCOUNT_ID" \
  --lesson-complete
python3 aws/cloudshell-deploy.py status
```

This requests app-stack deletion. Wait until that stack is ABSENT. Then repeat:

```
python3 aws/cloudshell-deploy.py cleanup \
  --confirm-delete-account "$STUDENT_AWS_ACCOUNT_ID" \
  --lesson-complete
python3 aws/cloudshell-deploy.py status
```

Wait for BOTH dedicated stacks to show ABSENT. CloudFront deletion can take time. Stopping one task is insufficient: ECS can replace it and the load balancer remains provisioned. Inspect deletion failures rather than assuming billing has stopped. Keep the helper's original configuration/state until deletion completes. The video's --video-verified flag remains an alias for --lesson-complete.

Package contents

- Dockerfile, .dockerignore, package.json and package-lock.json: build inputs.
- src, shared, server, fixtures and supporting files: application/lesson source.
- public: existing course images, infographics and audio.
- aws/build.template.json: private S3/ECR, CodeBuild and build logs.
- aws/app.template.json: CloudFront, ALB, ECS, task roles and logs.

- aws/cloudshell-deploy.py: account-aware deployment helper.
- aws/verify-source.py and aws/source-manifest.json: source checksums.
- IAM-REQUIREMENTS.md, TROUBLESHOOTING.md and VIDEO-CHAPTERS.txt: support material.

Installed dependencies, AWS CLI, instructor credentials, deployment state, recording evidence, authoring API keys and the MP4 are excluded. CodeBuild installs application dependencies. Watch the video in Udemy; this ZIP is its student resource.

Official references

- CloudShell authentication: https://docs.aws.amazon.com/cloudshell/latest/userguide/working-with-aws-cli.html
- CloudShell limits: https://docs.aws.amazon.com/cloudshell/latest/userguide/limits.html
- CodeBuild Docker: https://docs.aws.amazon.com/codebuild/latest/userguide/sample-docker.html
- ECS networking: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task-networking-awsvpc.html
- CloudFront/ALB access: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/restrict-access-to-load-balancer.html
- Fargate storage: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/fargate-task-storage.html

---

Permissions and account prerequisites

Ask your account administrator for a suitable approved training role or permission set. Organization policies, permission boundaries, region restrictions and quotas can still prevent provisioning. Do not use instructor credentials or grant AdministratorAccess simply to bypass an error.

The CloudShell identity needs permission for the helper's API calls and the resources CloudFormation creates/deletes. The templates are the authoritative inventory. This checklist is not a drop-in least-privilege IAM policy.

Operator permission areas

- STS: get caller identity.
- EC2/VPC: describe VPCs, subnets, route tables and managed prefix lists; manage the dedicated security groups/rules and their tags. Existing networking is not changed.
- Service Quotas: read the Fargate On-Demand vCPU quota.
- CloudFormation: validate, create, describe and delete the two stacks, inspect events and manage their purpose tags.
- S3: create/configure/tag/delete the dedicated bucket/policy, encryption, ownership and public-access block; upload/list/delete lesson source objects.
- ECR: create/read/tag/delete the repository and its images; describe image digests.
- CodeBuild: manage the dedicated project, start the build and read its status.

- CloudWatch Logs: manage the dedicated log groups/retention and read diagnostic logs.
- IAM: create/read/tag/delete service roles and inline policies; pass only the appropriate CodeBuild and ECS roles to those services. First use can require ECS or Elastic Load Balancing service-linked role creation.
- ECS: manage the dedicated cluster, task definition, service and deployment settings.
- Elastic Load Balancing v2: manage the ALB, target group, listener, rule, attributes/tags and inspect target health.
- CloudFront: create/read/update/tag/disable/delete the dedicated distribution.

CloudFront is global. A region-only organization policy may need a reviewed exception. CAPABILITY_IAM acknowledges role creation; it grants no missing permissions.

Created service roles

CodeBuild reads only the source bucket's source prefix, writes its build logs, and publishes to the dedicated repository. The execution role pulls this ECR image and writes application logs. ECR authorization-token retrieval requires its separate account-level action. The application task role has no AWS access policies. No AWS access key or provider API key belongs in the image.

Network and quota prerequisites

Use eu-north-1 with an existing default VPC, usable default subnets in at least two zones, active internet-gateway routes and quota for the one-vCPU task. Other workloads can consume that quota. A sandbox that blocks IAM roles, CodeBuild, CloudFront or public ALBs cannot run this exact architecture unchanged. Ask its owner rather than removing access protections.

Troubleshooting

Account configuration

Configure your own matching account before preflight. If deployment state already exists, keep its original account configuration. Changing a folder's account ID does not move AWS resources.

Upload or extraction

Check `df -h "$HOME"` and `ls -lh ~/*.zip`. Keep the source ZIP and extract only aws/*. Do not expand all media in CloudShell. If the filename acquired a suffix such as (1), adjust the source path or rename the upload before starting.

Budget or shell variable errors

Use a positive number with a decimal point. Shell variables disappear in a new terminal; restore the saved budget as shown in the guide. No estimate enforces a cap.

AccessDenied / UnauthorizedOperation

Report the exact denied action/resource to your training administrator. Use the templates and IAM-REQUIREMENTS.md as context. Preflight checks do not prove all creation permissions. Do not share tokens, access JSON, CloudFront configuration or listener rules: the latter can contain the private origin header.

Missing default network, routes or quota

Have the administrator prepare a suitable course account/network or deliberately adapt/review the templates. The helper does not create a default VPC. Fargate quota is regional; the recorded quota is not a promise about a new account.

Build resources still creating

Use status until the build stack reaches CREATE_COMPLETE. The first build call creates infrastructure; the second starts the image build. Avoid repeated creates.

Build running / CloudShell disconnected

Use status with the same saved buildId. BUILD and POST_BUILD are intermediate phases. Wait for SUCCEEDED before deploy. The cloud build continues after a CloudShell disconnect; return to the same helper folder and keep its state.

Build failed

Follow the build log deepLink from status. Inspect source download, manifest verification, npm installation, application build and ECR push separately. Do not modify the ZIP after checksums were generated. Additional build calls are blocked after a buildId is saved. Do not delete state to force a retry. After diagnosis, an experienced operator may explicitly retry the existing CodeBuild build with its project/source settings, or remove this dedicated deployment and start a reviewed clean attempt. Retrying is not polling.

App failure / rollback

Inspect the first failure in the dedicated CloudFormation stack Events tab, then ECS service events and the app log group if relevant. Check permissions, quotas, image digest/pull role, outbound network path and port 8080. The gateway starts its backend on loopback port 8081. The helper does not overwrite an existing app stack. Do not relax

security groups to bypass a startup error.

Browser 401 / 403 / 502

Anonymous 401 is expected. Run access privately and use its generated login at your own returned CloudFront HTTPS origin. Foreign Origin requests are deliberately denied. A 502 can indicate startup/backend failure: inspect target health/logs. Do not substitute a task IP or the ALB URL. The app password is not your AWS login.

HTTP checks disappear from evidence

Status refreshes evidence.json. Run verify again for current HTTP results. Neither command marks the manual browser exercises complete.

Notebook state

Use the same browser profile and hostname. Clearing site data or changing profiles changes browser-local notes. The template does not configure durable server data, database backups or a production student identity system.

Cleanup pending / failed

CloudFront deletion takes time. Poll until the app stack is ABSENT, then run cleanup again for the build stack. Resolve the exact failed deletion resource; do not assume a submitted delete stops all charges. Unexpected source-bucket objects intentionally block automatic removal. Preserve unrelated resources and keep your original helper/state until both stacks are ABSENT.

Offline helper checks

After extracting the full kit on a development machine, run:

```
python scripts/test-aws-deployment.py
```

These checks mock AWS. They verify account configuration and guards, state preservation, duplicate-build prevention, budget validation, cleanup guards and credential redaction. They create no AWS resources.